

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 638 — Natural Language Processing

Computing over a signal that carries meaning and does not carry structure

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 10 (with CSCE 676 and CSCE 670)

Prerequisites: CSCE 633 Machine Learning and CSCE 636 Deep Learning, both assumed throughout; CSCE 605 Compiler Design for the parsing material; CSCE 670 runs in parallel and shares Module 11

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A working language system on a task you chose, with a baseline it beats, an evaluation that measures the thing you care about rather than the thing that is easy, and a written account of the inputs on which it fails

Course description

Language is a hard input because it is **discrete, compositional, ambiguous at every level, and its structure is not given**. **Module 01** establishes those four properties properly, because every technique in the course is a response to one of them — subword tokenisation to discreteness, attention to composition, context to ambiguity, and pretraining to the missing structure.

The course follows the actual historical arc, because the arc is the argument. **Counting, then learned dense representations, then recurrence, then attention, then pretraining at scale** — and **each step removed a specific limitation that the previous step had made visible**, which is a more useful thing to understand than any single architecture.

The second theme is that **evaluation is the hard part and is routinely the weakest part of a language system**. **Module 10** is about why the standard metrics mislead — **a score that correlates poorly with the judgement you actually care about is worse than no score**, because it directs a year of work. This is CSCE 633 Module 12's argument, in the setting where it does the most damage.

The third is that **these systems fail in ways that matter to people**. **Module 12** covers **bias, distributional failure, and confident fabrication** — not as an appendix, but because **a system that produces fluent wrong answers is more dangerous than one that produces obviously wrong answers**, and fluency is exactly what these models optimise.

And the closing position is about what these systems do. **Module 13** is about stating that honestly — **a model that predicts the next token extremely well has demonstrated that, and nothing more**, and the

gap between that claim and the claims commonly made about such models is where this course asks you to be careful.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Explain the four properties that make language hard.
- 2 Tokenise and represent text, and explain what each choice costs.
- 3 Build and evaluate a language model.
- 4 Explain distributional word representations and their limits.
- 5 Explain recurrence, its failure mode, and what attention fixed.
- 6 Explain the transformer completely.
- 7 Explain pretraining and why transfer works here.
- 8 Adapt a pretrained model to a task under real constraints.
- 9 Explain decoding and why the strategy changes the output.
- 10 Evaluate a language system honestly.
- 11 Build a retrieval-augmented system and explain its failures.
- 12 Explain bias, distributional failure, and fabrication.
- 13 State what a language model's performance establishes.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	What Is Hard About Language	Why is text a harder input than an image?
2	Text as Data	What is a word, and does the question have an answer?
3	Language Models	What does a model of $P(\text{next} \mid \text{context})$ give you?
4	Word Representations	Can you learn what a word means from the company it keeps?
5	Sequence Models	How do you process a variable-length sequence?
6	The Transformer	What does attention compute, and why did it win?
7	Pretraining and Transfer	Why does a model trained on raw text help with your problem?
8	Adaptation	Full fine-tuning, a prompt, or something between?

Wk	Module	Core question it answers
9	Generation and Decoding	Why does the same model produce different quality text?
10	Evaluation	Does your number mean what you think it means?
11	Retrieval-Augmented Systems	How do you answer questions about your own documents?
12	Bias, Failure, and Fabrication	What goes wrong, and for whom?
13	Claiming Language Understanding Honestly	The model scored well. What follows?

Projects

Project 1 — A language model, built and probed (after Module 6)

Build a language model from counting through to attention, and measure what each step bought.

Requirements

- **An n-gram model with smoothing**, evaluated by perplexity on held-out text (Module 03 §2).
- **A neural language model** with learned embeddings, compared on the same held-out set.
- **A recurrent model**, and **a demonstration of its long-range failure** (Module 05 §2) on a task you construct.
- **A single attention layer added**, and the same task re-measured.
- **A written account of what each step bought**, with the numbers and with the cases that changed.
- **And three inputs your best model handles badly**, with an explanation of why.

Done means

- **All four models evaluated on the identical held-out split**, which is the only way the comparison means anything.
- **The long-range failure demonstrated rather than asserted** — **you constructed a task that isolates it**, and that construction is graded hardest.
- **The improvements attributed correctly**, including where a step bought you nothing, which happens and should be reported.
- **And the three failures explained mechanically**, not as 'the model is imperfect'.

Project 2 — A language system, honestly evaluated (after Module 12)

Build a system for a task you care about, and evaluate it as if somebody were going to rely on it.

Requirements

- **A task and a stated decision it supports** — **what would someone do differently** given the output?
- **A baseline that is genuinely hard to beat** — **majority class, retrieval, or a well-tuned simple model**, not a strawman (Module 10 §2).
- **Your system, with the adaptation method justified** against its cost (Module 08).
- **An evaluation measuring the thing you care about**, with the metric's correlation to that thing argued rather than assumed.
- **An error analysis by category**, with at least twenty errors read individually.
- **And a failure and limitation statement** covering distribution shift, bias, and fabrication (Module 12).

Done means

- **The baseline strong — a system that beats a strawman has established nothing**, and this is graded first.
- **The metric justified against the decision**, which is the project's central requirement and the one most often skipped.
- **Twenty errors actually read** — the categories must come from the data rather than from your expectations.
- **And the limitation statement specific and non-empty**, in Module 13 §2's form. **A system reported without known failures has not been evaluated.**

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Jurafsky & Martin — Speech and Language Processing, 3rd edition (free draft)	The primary text, free in full from the authors, and the best book in the field. It covers the classical and the neural material with equal care, which is unusual and is exactly what this course needs.
Eisenstein — Introduction to Natural Language Processing (free draft)	The rigorous companion. Stronger on the learning theory and the structured prediction material, and free as a draft PDF.
Goldberg — Neural Network Methods for NLP	Modules 04 through 06. Written before the transformer took over and consequently much clearer about <i>why</i> each architectural choice was made. Library copy.
The Illustrated Transformer, and The Annotated Transformer (free)	Module 06 twice over — once as a diagram and once as working code. Read the illustrated version first, then implement from the annotated one.
Bender et al. and the model-documentation literature (free)	Modules 12 and 13. The paper that framed the scale-and-harm argument, plus the model card and datasheet practices that came out of it.

Resource	Role
Hugging Face course and documentation (free)	The practical toolchain for Modules 07 through 11 — free, current, and the implementations every exercise in the second half uses.

Tooling

- **Python with PyTorch**, continuing from CSCE 636 — **and the exercises assume you can still write a training loop by hand**, because Module 05's gradient behaviour is only visible from inside one.
- **Hugging Face transformers, tokenizers, and datasets** for Modules 07 onward. **Read the tokeniser library's output rather than trusting it**, which is Module 02 §4's exercise.
- **spaCy or stanza** for the classical pipeline in Modules 01 through 03 — **worth using once to see what a parser produces**, even though you will not build one.
- **A GPU, or free hosted notebooks**. Every exercise is sized for a single modest GPU or a free tier, and Module 08's parameter-efficient methods exist partly so that this is possible.
- **A small pretrained model you can fine-tune in minutes** — **the pedagogy requires iteration speed more than capability**, so prefer the smallest model that shows the effect.
- **And a held-out test set you look at once**. **Module 10 §1's discipline is the one habit from this course most worth keeping**, and it has to be set up before you start rather than after.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.